

id3

January 10, 2026

B.3. Write Python code to implement the ID3 algorithm and test it on the PlayTennis dataset, verifying the inductive bias of the decision tree learning algorithm.

```
[4]: import pandas as pd
import numpy as np
import math
```

```
[2]: data = pd.read_csv("PlayTennis.csv")
df = pd.DataFrame(data)
df
```

```
[2]:
```

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

ID3 (Iterative Dichotomiser 3) builds a decision tree by:

1. Looking at all features
2. Measuring how good each feature is at splitting the data
3. Picking the best feature
4. Splitting the dataset on that feature
5. Repeating this process recursively

Now one of the most important concepts of ID3 is Entropy so lets start by understanding that.

Entropy

Formula: $Entropy(S) = -\sum p_i \log_2 p_i$

Intuition

1. Entropy = 0 $\rightarrow$ perfectly pure (all Yes or all No)
2. Entropy = 1 $\rightarrow$ very mixed

So, Entropy basically tells **"How hard is it to guess the class label?"**

Easy to guess $\rightarrow$ low entropy

Hard to guess $\rightarrow$ high entropy

Entropy measures label impurity. $\rightarrow$ Lower entropy = better split.

```
[5]: def entropy(labels):  
    total = len(labels)  
    count = {}  
  
    for label in labels:  
        count[label] = count.get(label, 0) + 1  
  
    ent = 0  
  
    for count in count.values():  
        p = count / total  
        ent -= p*math.log2(p)  
  
    return ent
```

```
[13]: entropy(list(df.PlayTennis))
```

```
[13]: 0.9402859586706311
```

Entropy of 0.94 is quite mixed so we need to split it and form a decision tree

Information Gain (IG)

Information Gain (IG) answers this question: *If I split my data using this feature, how much uncertainty*

1. Big reduction in uncertainty $\rightarrow$ high information gain
2. Small reduction $\rightarrow$ low information gain

$$\text{Formula} = IG(S, A) = Entropy(S) - \sum_{v \in A} \frac{|S_v|}{|S|} Entropy(S_v)$$

Where:

1. S = full dataset
2. A = feature
3. v = a value of that feature
4. S_v = subset where feature = v

```
[6]: def information_gain(data, feature_index, target_index):  
    total_entropy = entropy([row[target_index] for row in data])  
    total = len(data)
```

```

feature_values = {}
for row in data:
    value = row[feature_index]
    feature_values.setdefault(value, []).append(row)

weighted_entropy = 0
for subset in feature_values.values():
    subset_labels = [row[target_index] for row in subset]
    weighted_entropy += (len(subset) / total) * entropy(subset_labels)

return total_entropy - weighted_entropy

```

```

[16]: def majority_label(labels):
        return max(set(labels), key=labels.count)

```

```

[14]: data = df.values.tolist()
features = list(range(df.shape[1] - 1)) # all columns except target
target_index = df.shape[1] - 1

```

ID3 picks the feature with maximum information gain.

```

[25]: def best_feature(data, features, target_index):
        best_gain = -1
        best_feat = None

        for feature in features:
            gain = information_gain(data, feature, target_index)
            if gain > best_gain:
                best_gain = gain
                best_feat = feature

        return best_feat

```

```

[26]: bf = best_feature(data, features, target_index)

```

```

[27]: bf

```

```

[27]: 0

```

Build the tree

```

[28]: def build_tree(data, features, target_index):
        labels = [row[target_index] for row in data]

        # Stop 1: all labels same
        if labels.count(labels[0]) == len(labels):
            return labels[0]

```

```

# Stop 2: no features left
if not features:
    return majority_label(labels)

# Choose best feature
best_feat = best_feature(data, features, target_index)
tree = {best_feat: {}}

# Split on best feature
feature_values = set(row[best_feat] for row in data)

for value in feature_values:
    subset = [row for row in data if row[best_feat] == value]
    remaining_features = [f for f in features if f != best_feat]

    tree[best_feat][value] = build_tree(
        subset,
        remaining_features,
        target_index
    )

return tree

```

```
[29]: tree = build_tree(data, features, target_index)
```

```
[30]: tree
```

```
[30]: {0: {'Sunny': {2: {'High': 'No', 'Normal': 'Yes'}},
        'Overcast': 'Yes',
        'Rain': {3: {'Strong': 'No', 'Weak': 'Yes'}}}}
```

```
[31]: feature_names = df.columns[:-1]
```

```

def pretty_print(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "→", tree)
        return

    for feature, branches in tree.items():
        print(indent + feature_names[feature])
        for value, subtree in branches.items():
            print(indent + f" [{value}]")
            pretty_print(subtree, indent + "    ")

```

```
[32]: pretty_print(tree)
```

```

Outlook
[Sunny]

```

```
Humidity
  [High]
    → No
  [Normal]
    → Yes
[Overcast]
  → Yes
[Rain]
  Wind
    [Strong]
      → No
    [Weak]
      → Yes
```

```
[24]: def predict(tree, sample):
        if not isinstance(tree, dict):
            return tree

        feature = next(iter(tree))
        value = sample[feature]

        return predict(tree[feature][value], sample)
```

```
[33]: data[0]
```

```
[33]: ['Sunny', 'Hot', 'High', 'Weak', 'No']
```

```
[34]: sample = data[0]
        print("Prediction:", predict(tree, sample))
        print("Actual:", sample[target_index])
```

```
Prediction: No
Actual: No
```

```
[39]: correct = 0

        for row in data:
            print(f"The Prediction row: {row}")
            prediction = predict(tree, row)
            actual = row[target_index]

            if prediction == actual:
                correct += 1

        accuracy = correct / len(data)
        accuracy
```

```
The Prediction row: ['Sunny', 'Hot', 'High', 'Weak', 'No']
The Prediction row: ['Sunny', 'Hot', 'High', 'Strong', 'No']
```

```

The Prediction row: ['Overcast', 'Hot', 'High', 'Weak', 'Yes']
The Prediction row: ['Rain', 'Mild', 'High', 'Weak', 'Yes']
The Prediction row: ['Rain', 'Cool', 'Normal', 'Weak', 'Yes']
The Prediction row: ['Rain', 'Cool', 'Normal', 'Strong', 'No']
The Prediction row: ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes']
The Prediction row: ['Sunny', 'Mild', 'High', 'Weak', 'No']
The Prediction row: ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes']
The Prediction row: ['Rain', 'Mild', 'Normal', 'Weak', 'Yes']
The Prediction row: ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes']
The Prediction row: ['Overcast', 'Mild', 'High', 'Strong', 'Yes']
The Prediction row: ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes']
The Prediction row: ['Rain', 'Mild', 'High', 'Strong', 'No']

```

[39]: 1.0

```

[38]: for i, row in enumerate(data):
        print(
            f"Day {i+1}: "
            f"Predicted = {predict(tree, row)}, "
            f"Actual = {row[target_index]}"
        )

```

```

Day 1: Predicted = No, Actual = No
Day 2: Predicted = No, Actual = No
Day 3: Predicted = Yes, Actual = Yes
Day 4: Predicted = Yes, Actual = Yes
Day 5: Predicted = Yes, Actual = Yes
Day 6: Predicted = No, Actual = No
Day 7: Predicted = Yes, Actual = Yes
Day 8: Predicted = No, Actual = No
Day 9: Predicted = Yes, Actual = Yes
Day 10: Predicted = Yes, Actual = Yes
Day 11: Predicted = Yes, Actual = Yes
Day 12: Predicted = Yes, Actual = Yes
Day 13: Predicted = Yes, Actual = Yes
Day 14: Predicted = No, Actual = No

```

Observation 1: Root node selection

```

[47]: bf = best_feature(data, features, target_index)
        print(f"Best Feature = {list(df.keys())[bf]}")

```

Best Feature = Outlook

Why Outlook is is the best Feature?

The feature Outlook has three possible values:

- Sunny
- Overcast
- Rain

When you split the dataset on Outlook, you get: * Outlook = Sunny * Mostly No * Very low uncertainty

- Outlook = Overcast
 - All Yes
 - Zero uncertainty (perfectly pure)
- Outlook = Rain
 - Mostly Yes
 - Low uncertainty

So after the split: * One group is perfectly pure * The other two are almost pure

Very little confusion remains

This means the average uncertainty after splitting is very low.

Let's compare this with another feature, like Temperature or Wind.

When you split on those:

Each resulting group still contains a mix of Yes and No

No group becomes perfectly pure

You still have significant uncertainty after the split

So although they reduce entropy a bit, they do not reduce it as much as Outlook does.

Observation 2: Tree structure is shallow

Your printed tree shows: * Outlook at the root * At most two levels of depth

Several branches terminate early with pure labels This reflects the assumption that:

A smaller, simpler tree is preferable to a larger one.

ID3 does not explore deeper trees if entropy is already minimized.

Why this is inductive bias and not “generalization”?

The dataset used for testing is the training dataset Therefore, the accuracy measures training performance, not generalization

This highlights another aspect of ID3's inductive bias:

ID3 does not protect against overfitting It assumes the training data is representative

No pruning is performed

The algorithm is biased toward consistency with training data, even if that leads to overfitting.